

# **FIAP - FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA**

Instituto especializada em tecnologia e gestão empresarial

Pós Tech de Software Architecture

Augusto Zanesco Bortoncello

Claudio Gabriel Kosooski

Daniel da Conceição Alferes

Fernando Oliveira Soares

Oéslei Ismael Kuhn

## **Tech Challenge - Visão geral do sistema**

Sistema de Gestão de Oficina Mecânica

**Grupo Archflow**

Digital e hands-on

Turma Online

2026



## **FIAP - FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA**

Pós Tech de Software Architecture

### **FASE 1**

Augusto Zanesco Bortoncello - RM374903

Claudio Gabriel Kosooski - RM375013

Daniel da Conceição Alferes - RM374468

Fernando Oliveira Soares - RM374940

Oéslei Ismael Kuhn - RM374924

### **Tech Challenge - Visão geral do sistema**

Sistema de Gestão de Oficina Mecânica

Trabalho de pós tech apresentado ao curso de Software Architecture, como parte dos requisitos necessários à obtenção do título de conclusão da fase 1.

Orientadores: Colaborativa por todo o corpo de professores do ecossistema FIAP.

### **Grupo Archflow**

Turma Online

2026

*“Se você não jogar, jamais irá vencer”.*  
*Charles Bukowski*

## RESUMO

Este documento descreve a visão geral do sistema de gestão de oficina mecânica desenvolvido no Tech Challenge da FIAP PosTech. A solução é uma API REST em PHP 8.4 com Slim Framework 4, seguindo princípios de DDD. Contempla gestão de clientes, veículos, peças, serviços, estoque e ordens de serviço, com autenticação JWT e notificações por e-mail. A aplicação é containerizada com Docker, utiliza MySQL 9 como banco de dados e conta com pipeline de CI/CD via GitHub Actions e análise de qualidade pelo SonarCloud.

**Palavras-Chave:** API REST; PHP 8.4; Slim Framework; DDD; Docker; MySQL; JWT; Oficina Mecânica.

## ABSTRACT

This document provides an overview of a mechanic shop management system developed for the FIAP PosTech Tech Challenge. The solution is a REST API built with PHP 8.4 and Slim Framework 4, following DDD principles. It covers customer, vehicle, parts, service, inventory, and service order management, with JWT authentication and email notifications. The application is containerized with Docker, uses MySQL 9 as the database, and includes a CI/CD pipeline via GitHub Actions with code quality analysis by SonarCloud.

**Keywords:** REST API; PHP 8.4; Slim Framework; DDD; Docker; MySQL; JWT; Mechanic Shop Management.

## LISTA DE ABREVIATURAS E SIGLAS

| <b>Sigla</b> | <b>Descrição</b>                              |
|--------------|-----------------------------------------------|
| API          | Interface de Programação de Aplicação         |
| CI/CD        | Integração Contínua / Entrega Contínua        |
| CRUD         | Criar, Ler, Atualizar e Deletar               |
| DDD          | Design Orientado ao Domínio                   |
| DI           | Injeção de Dependências                       |
| DTO          | Objeto de Transferência de Dados              |
| HLD          | Design de Alto Nível                          |
| HTTP         | Protocolo de Transferência de Hipertexto      |
| JSON         | Notação de Objeto JavaScript                  |
| JWT          | Token Web JSON                                |
| OS           | Ordem de Serviço                              |
| OWASP        | Projeto Aberto de Segurança em Aplicações Web |
| PDO          | Objetos de Dados PHP                          |
| PHP          | Pré-processador de Hipertexto                 |
| PSR          | Recomendação de Padrões PHP                   |
| REST         | Transferência de Estado Representacional      |
| SMTP         | Protocolo Simples de Transferência de Correio |
| SQL          | Linguagem de Consulta Estruturada             |
| SRP          | Princípio da Responsabilidade Única           |
| URL          | Localizador Uniforme de Recursos              |
| ZAP          | Proxy de Ataque Zed                           |

## Sumário

|      |                                     |    |
|------|-------------------------------------|----|
| 1.   | Objetivo do Sistema.....            | 10 |
| 2.   | Atores e Usuários .....             | 11 |
| 3.   | Diagrama de Alto Nível (HLD) .....  | 12 |
| 4.   | Principais Componentes.....         | 13 |
| 4.1. | Containerização Docker .....        | 13 |
| 4.2. | Servidor Web Nginx .....            | 13 |
| 4.3. | Aplicação PHP Slim Framework .....  | 13 |
| 4.4. | Banco de Dados MySQL 9.....         | 13 |
| 4.5. | Serviços Externos .....             | 14 |
| 5.   | Módulos Funcionais.....             | 15 |
| 6.   | Segurança .....                     | 17 |
| 7.   | Stack Tecnológica .....             | 18 |
| 8.   | Fluxo Macro de uma Requisição ..... | 20 |
| 9.   | Conclusão.....                      | 22 |
| 10.  | Bibliografia.....                   | 23 |

## Índice de Ilustrações

|                                                            |    |
|------------------------------------------------------------|----|
| Figura 1 - HLD: Sistema de Gestão de Oficina Mecânica..... | 12 |
|------------------------------------------------------------|----|



## Lista de Tabelas

|                                                |    |
|------------------------------------------------|----|
| Tabela 1 - Atores e Usuários .....             | 11 |
| Tabela 2 - Serviços Externos.....              | 14 |
| Tabela 3 - Módulos Funcionais.....             | 15 |
| Tabela 4 - Stack Tecnológica .....             | 18 |
| Tabela 5 - Fluxo Macro de uma Requisição ..... | 20 |

## 1. Objetivo do Sistema

O sistema de gestão de oficina mecânica tem como objetivo centralizar e automatizar os processos operacionais de uma oficina, cobrindo o cadastro de clientes e veículos, controle de estoque de peças, catálogo de serviços e o ciclo de vida completo das Ordens de Serviço.

A solução é exposta como uma API REST, consumível por qualquer frontend (web ou mobile) ou sistema integrador, com autenticação segura via JWT e notificações automáticas por e-mail ao cliente.

## 2. Atores e Usuários

*Tabela 1 - Atores e Usuários*

| <b>Ator</b>                                 | <b>Perfil</b>                  | <b>Interação com o Sistema</b>                                       |
|---------------------------------------------|--------------------------------|----------------------------------------------------------------------|
| Mecânico /<br>Recepcionista /<br>Almoxarife | Usuário interno<br>autenticado | Opera todas as funcionalidades via API<br>REST (CRUD, O.S., estoque) |
| Cliente (Dono do<br>Veículo)                | Usuário externo<br>passivo     | Recebe notificações por e-mail sobre o<br>andamento da O.S.          |
| Sistema CI/CD                               | Agente automatizado            | Executa testes, análise de qualidade e<br>build via GitHub Actions   |

*Fonte: Autoria própria*



## 4. Principais Componentes

### 4.1. Containerização Docker

Toda a solução é executada em containers Docker orquestrados via Docker Compose, garantindo isolamento de ambiente, portabilidade e reprodutibilidade. Os serviços containerizados são: Nginx (servidor web), PHP-FPM (runtime da aplicação) e MySQL 9 (banco de dados).

### 4.2. Servidor Web Nginx

Atua como ponto de entrada único de todas as requisições HTTP. Funciona como proxy reverso, encaminhando as requisições dinâmicas ao PHP-FPM via protocolo FastCGI. Também é responsável por direcionar todas as rotas ao front controller da aplicação (index.php).

### 4.3. Aplicação PHP Slim Framework

Núcleo da solução, construído com PHP 8.4 e Slim Framework 4. Segue uma arquitetura em camadas inspirada em DDD (Domain-Driven Design):

- **Router:** Resolve e despacha as requisições HTTP às funções corretas.
- **Middlewares:** Interceptam requisições para tratar autenticação JWT e erros globais.
- **Controllers:** Recebem a requisição, extraem e validam dados de entrada e delegam ao Service.
- **Services:** Concentram as regras de negócio, orquestrando as operações de domínio.
- **Repositories:** Abstraem o acesso ao banco de dados, isolando a infraestrutura do domínio.

### 4.4. Banco de Dados MySQL 9

Banco de dados relacional responsável pela persistência de todas as entidades do sistema: clientes, veículos, peças, serviços, estoque e ordens de

serviço. O acesso é feito exclusivamente via PDO com prepared statements, prevenindo SQL Injection.

## 4.5. Serviços Externos

*Tabela 2 - Serviços Externos*

| <b>Serviço</b>         | <b>Função</b>                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------|
| SMTP Server            | Envio de notificações automáticas por e-mail ao cliente (abertura, andamento e conclusão de O.S.) |
| GitHub Actions (CI/CD) | Pipeline automatizado de testes, build e análise de qualidade a cada commit                       |
| SonarCloud             | Análise estática de código: bugs, code smells, duplicações e cobertura de testes                  |

*Fonte: Autoria própria*

## 5. Módulos Funcionais

A aplicação está organizada em módulos funcionais que refletem os subdomínios do negócio da oficina mecânica.

*Tabela 3 - Módulos Funcionais*

| <b>Módulo</b> | <b>Responsabilidade</b>                                               | <b>Tipo</b> |
|---------------|-----------------------------------------------------------------------|-------------|
| Autenticação  | Login de usuários e emissão de tokens JWT para acesso à API           | Público     |
| Clientes      | Cadastro e gestão dos donos de veículos atendidos pela oficina        | Protegido   |
| Veículos      | Cadastro e gestão dos veículos vinculados aos clientes                | Protegido   |
| Peças         | Catálogo de peças utilizadas nos serviços da oficina                  | Protegido   |
| Serviços      | Catálogo dos tipos de serviços prestados com preços e estimativas     | Protegido   |
| Estoque       | Controle de inventário de peças, incluindo entradas e baixas de itens | Protegido   |

|                   |                                                                                       |           |
|-------------------|---------------------------------------------------------------------------------------|-----------|
| Ordens de Serviço | Ciclo completo de atendimento: abertura, execução, conclusão e notificação ao cliente | Protegido |
|-------------------|---------------------------------------------------------------------------------------|-----------|

*Fonte: Autoria própria*



## 6. Segurança

A segurança da API foi projetada seguindo as diretrizes do OWASP Top 10, com varredura ativa realizada pelo OWASP ZAP. Os principais controles adotados são:

- Autenticação via JWT (JSON Web Token) com expiração configurável.
- Todas as rotas operacionais protegidas por middleware de autenticação.
- Senhas armazenadas com hash bcrypt, nunca em texto plano.
- Queries ao banco de dados com PDO e Prepared Statements (prevenção de SQL Injection).
- Código analisado continuamente pelo SonarCloud para identificação de security hotspots

## 7. Stack Tecnológica

*Tabela 4 - Stack Tecnológica*

| <b>Camada</b>            | <b>Tecnologia</b>       | <b>Tipo</b>     |
|--------------------------|-------------------------|-----------------|
| Linguagem de Programação | PHP                     | 8.4             |
| Framework Web            | Slim Framework          | 4.x             |
| Banco de Dados           | MySQL                   | 9               |
| Servidor Web             | Nginx                   | Alpine (latest) |
| Containerização          | Docker / Docker Compose | —               |
| Autenticação             | JWT (firebase/php-jwt)  | —               |
| Testes                   | PHPUnit                 | —               |
| Análise de Código        | SonarCloud              | —               |

|        |                  |   |
|--------|------------------|---|
| CI/CD  | GitHub Actions   | — |
| E-mail | PHPMailer (SMTP) | — |

*Fonte: Autoria própria*

## 8. Fluxo Macro de uma Requisição

O diagrama abaixo ilustra o caminho percorrido por uma requisição autenticada de ponta a ponta no sistema.

*Tabela 5 - Fluxo Macro de uma Requisição*

| <b>Etapa</b> | <b>Componente</b>   | <b>Descrição</b>                                         |
|--------------|---------------------|----------------------------------------------------------|
| 1            | Cliente HTTP        | Envia requisição com token JWT no header                 |
| 2            | Nginx               | Recebe e encaminha ao PHP-FPM via FastCGI                |
| 3            | Middlewares         | Valida o JWT, erros são capturados globalmente           |
| 4            | Router + Controller | Roteia para o módulo correto, válida os dados de entrada |
| 5            | Service             | Aplica as regras de negócio do domínio                   |
| 6            | Repository → MySQL  | Persiste ou consulta os dados no banco                   |
| 7            | Service (evento)    | Se aplicável, dispara notificação via SMTP               |

|   |            |                                                |
|---|------------|------------------------------------------------|
| 8 | Controller | Retorna resposta JSON com status HTTP adequado |
|---|------------|------------------------------------------------|

*Fonte: Autoria própria*

## 9. Conclusão

O sistema de gestão de oficina mecânica foi concebido com foco em organização, manutenibilidade e segurança. A escolha do Slim Framework sobre PHP 8.4 permite uma base leve e de alta performance, enquanto a arquitetura em camadas inspirada em DDD garante que cada componente tenha responsabilidade bem definida e possa evoluir de forma independente.

A containerização com Docker elimina inconsistências entre ambientes, tornando o processo de desenvolvimento, testes e implantação previsível e reproduzível. O pipeline de CI/CD integrado ao GitHub Actions assegura que qualidade e segurança sejam verificados automaticamente a cada mudança no código, com o SonarCloud atuando como guardião contínuo das métricas de qualidade.

As integrações com serviços externos SMTP para notificações e OWASP ZAP para varredura de segurança completam uma solução alinhada às boas práticas da indústria e aos requisitos do Tech Challenge FIAP PosTech.Conclusão.

## 10. Bibliografia

1. INC, D. dockerdocs. **https:** //docs.docker.com/, 2013-2026. Disponível em: <<https://docs.docker.com/>>. Acesso em: 16 Mai 2026.
2. GROUP, P. PHP 8.4. **PHP**, 2001-2026. Disponível em: <<https://www.php.net/releases/8.4/en.php>>. Acesso em: 09 Mai 2026.
3. STOCK, A. V. D. OWASP. **OWASP**, 2025. Disponível em: <<https://owasp.org/www-project-top-ten/>>. Acesso em: 30 Mai 2026.
4. DATENSCHUTZ, I. J. PHPUnit. **PHPUnit**, 2026. Disponível em: <<https://phpunit.de/>>. Acesso em: 07 Jun 2026.